

RegEx

Introduzione

Una RegEx è la forma abbreviata di espressione regolare: **è un insieme di simboli (stringhe) che permette di identificare un gruppo specifico di stringhe**, quindi permettono di:

- **Trovare** testo corrispondente a uno schema.
- **Validare** formati (email, codici fiscali, ecc.).
- **Estrarre** o **sostituire** parti di stringhe.
identificare un pattern o schemi all'interno del testo.

Casi d'uso delle RegEx

Il caso d'uso: supponiamo di avere un testo e siamo alla ricerca di specifiche parole, usiamo una RegEx, quindi si fanno dei match.

Quindi ad esempio:

- per trovare le email
- estrarre numeri(solo stringhe quindi stringhe numeriche)
- identificare parole specifiche

Le RegEx sono importanti perché sono supportati da Python, hanno una libreria dedicata, ma sono importanti anche in progettazione(es: codice fiscale stringa numerica secondo standard, in questo caso si usano le espressioni regolari).

In questo caso però ci servono per fare un match nelle stringhe.

Operatori delle RegEx

Per costruire una RegEx abbiamo diversi operatori:

Ancore

Le ancorre si suddividono in:

^ - Start -of-line anchor:

Matchano la posizione all'inizio della stringa o della linea.

Esempio:

Se scriviamo `^Hello`, matcha ogni linea che inizia con "Hello", quindi, ad esempio, il match è con `"Hello World"` ma non `"Oh Hello"`.

In altre parole stiamo cercando tutte le linee che iniziano con quella parola.

\$ - **End-of-line:**

Matchano la posizione alla fine della stringa o della linea .

Esempio: `world$` matcha ogni linea che finiscono con "world".

Cioè se si ha la stringa "Hello word" avverrà il match, ma se si ha la stringa "worldwide" no perché `world` deve essere messo a fine frase.

In altre parole stiamo cercando tutte le linee che finiscono con quella parola

Se invece scrivo `^Hello$` : stiamo esattamente cercando la parola Hello .

I quantificatori:

I quantificatori permettono di definire e stabilire il numero di occorrenze in una stringa.

Esistono diversi tipi di quantificatori:

- `*` : indica lo zero o più occorrenze dell'elemento che precede.

Esempio:

se scriviamo `abc*` :

vuol dire che in questo caso agisce solo su c non su ab, quindi matchia "ab", "abc", "abcc", "abccc", etc.

- `+` : uno o più, matchia 1 o più occorrenze dell'elemento precedente.

Esempio:

Se scriviamo `abc+` ; matchia "abc", "abcc", "abccc" , ma non "ab".

- `?` :

da zero a uno, matchia da 0 a 1 occorrenza dell'elemento che precede

Esempio:

Se scriviamo `abc?` ; matchia "ab" o "abc" .

Con questi primi quantificatori andiamo a definire il numero di occorrenze, mentre se volessimo definire il numero di occorrenze:

- `{n}` : **Esattamente n volte.**

Ovvero esprime esattamente le occorrenze n volte, esattamente per quante volte possono servire.

Agisce sempre sul carattere antecedente

Esempio:

se scriviamo `abc{3}` ; matchia "abccc" poiché matcherà con le occorrenze di c che si ripetono per tre volte nella stessa stringa.

- `{n,}` : **almeno n volte** ,

Esempio:

se scriviamo `abc{2,}` ; il match corrisponde a "abcc", "abccc", "abccccc", etc.

- `{n,m}` : tra n a m volte ,

Esempio:

Se scriviamo `abc{2,4}` ; matchia le corrispondenze di "c" da 2 a 4 volte.

Ovvero matchia "abcc", "abccc", "abccccc".

Wildcards

Le **wildcard** (o metacaratteri) nelle espressioni regolari sono: **simboli speciali che rappresentano uno o più caratteri generici, permettendo di matchare pattern flessibili all'interno di un testo.**

- `.` : significa qualsiasi singolo carattere eccetto la newline (`\n`) di default, ad esempio se scriviamo `a.c` matchiamo "axc", "alc", "a c", "a-c", "a;c", ma non "ac" (manca un carattere) o "abbc" (troppi caratteri).
- `.*` : da zero a più di qualsiasi caratteri

matcha qualsiasi sequenza di caratteri (anche vuota).

Esempio: se scriviamo `a*c` questo matcha:

- "abc",
- "xyz",
- "a123xzc"

In somma da `a` a qualsiasi carattere a `c` , cioè il match trova da qualsiasi carattere a partire da `a` a `c` .

- `.+` : **matcha da almeno uno a più caratteri qualsiasi** .

Simile a `.*` ma **richiede almeno un carattere tra quelli specificati.**

Esempio: se scriviamo `a.+c` matcha

- "abc",
- "xyz"
- ma non "ac" perché manca un carattere nel mezzo

❓ Differenza tra Wildcard e Quantificatori ▾

- **Wildcard** (come `.`):
definiscono cosa può essere matchato (es: "qualsiasi carattere").
- **Quantificatori** (come `*` , `+` , `?` , `{n}`):
definiscono quante volte può apparire.
 - Esempio: `a.*c` usa sia la wildcard (`.`) che il quantificatore (`*`).

Operatori logici delle RegEx

1. `|` (l'alternazione OR): **matcha l'espressione prima o l'espressione dopo o entrambe se scritte attaccate all'operatore**.

≡ Esempio ▾

- Se scriviamo `a|b` -> matcha "a" oppure "b".
- Stessa cosa se scriviamo `cat|dog` -> matcha "cat" o "dog"
- Quando viene usato in un gruppo; `(abc|def)` -> matcha "abc" o "def".

2. `(...)` : raggruppa le espressioni e le cattura matchando il testo.

≡ Esempio

`(abc)+` matcha -> "abc", "abcabc", etc.

3. `(?:)` : il **non-capture group** costruisce il gruppo ma non lo salva per poterlo riutilizzare dopo.

Per fare maggiore chiarezza:

supponiamo si voglia utilizzare un pattern di "abc":

con il **Capture group** se si scrive : `(abc){3}` -> restituisce "abcabcabc", in questo caso il capture cattura (salva in memoria) l'ultimo gruppo matchato ("abc").

La RegEx lo verifica che sia tre volte ma lo salva come "abc" e quindi si può agire sul gruppo del pattern.

In sostanza l'output sarà "abc".

Con il solo **Non-capture group** sarà: `(?:abc){3}` -> che restituisce "abcabcabc", formalmente descriviamo una stringa che si ripete 3 volte ma all'atto pratico lo va vedere come `['abcabcabc']`.

Questo perché - La regex cerca ugualmente "abc" ripetuta 3 volte ("abcabcabc") ma **non cattura** nulla, verifica solo il pattern.

Lookaround(lookahead/lookbehind)

I lookahead sono costrutti avanzati che controllano il contesto prima o dopo una certa porzione di testo, senza includerla nel match.

In sostanza aiutano a filtrare senza catturare

Lookahead

1. (?=...) : Positive lookahead

Verifica che ciò che segue la posizione corrente corrisponda all'espressione .

In altre parole:

≡ Esempio

Se scriviamo `a(?=b)` ; matcha a solo se seguita da b

2. (?!...) : Negative lookahead

Verifica che ciò che segue la posizione corrente non corrisponda all'espressione.

≡ Esempio

Se scriviamo `a(?!b)` ; matcha "a" solo se non è seguita da "b".

Lookbehind

3. (?<=...) : Positive lookbehind

Verifica che ciò che sta prima della posizione corrente corrisponda all'espressione

≡ Esempio

Se scriviamo `(?<=Y)X` ; matcha "x" solo se è preceduto da "y"

4. (?<!=...) : Negative lookbehind

Verifica che ciò che sta prima della posizione corrente non corrisponda all'espressione.

≡ Esempio

Se scriviamo `(?<!=Y)X` ; matcha x solo se non è preceduto da y

Grouping/Capture

Il **grouping** (raggruppamento) e il **capture** (cattura) sono meccanismi delle espressioni regolari che permettono di:

1. **Raggruppare parti di un pattern per applicare quantificatori (+ , * , {n}) a un intero blocco.**
2. **Catturare il testo matchato per riutilizzarlo successivamente (es: backreference, estrazione).**

🔗 Cosa cambia tra il grouping con capture e il solo grouping a livello pratico?

Il grouping con capture cambia con il solo grouping a livello pratico:

1. **Grouping con capture (. . .) :**
 - **Oltre a verificare il match, salva il gruppo in memoria.**
 - Quando utilizzarlo:
 - **Riutilizzare** il testo catturato (es: con backreference `\1` , `\2`).
 - **Estrarre** il gruppo matchato (es: in Python con `.group(1)`).
2. **Solo grouping (Non-Capture group, (?: . . .))**
 - **Verifica il match senza salvare il gruppo.**
 - Quando utilizzarlo:
 - Applicare un quantificatore a un blocco (es: `(?:abc)+`).
 - Ottimizzare la regex (meno overhead di memoria).

✍ Cos'è un *non-capturing group*?

Un non-capturing group è una forma di raggruppamento che serve solo a strutturare la regex, senza memorizzare il contenuto matchato.

Sintassi: `(?: . . . )`

È utile quando hai bisogno solo della struttura, ad esempio per usare quantificatori o alternanze,

ma non ti interessa riutilizzare o estrarre quel gruppo .

Classi di caratteri

Le **classi di caratteri** (o *character classes*) sono uno strumento nelle espressioni regolari che **ti permettono di specificare un insieme di caratteri possibili per**

matchare un singolo carattere in una stringa.

Sono racchiuse tra **parentesi quadre** `[]` e sono estremamente utili per cercare pattern flessibili.

☰ Esempio

`gr[ae]y` → matcha **"gray"** o **"grey"** (solo un carattere tra `a` ed `e`)

Ci sono delle classi caratteri di caratteri comuni:

Ranges:

- `[a-z]` : **matcha qualsiasi lettera minuscola dalla a alla z di un solo carattere alla volta (esclude i caratteri speciali)** .
- `[0-9]` : **matcha una cifra da 0 a 9 alla volta** .

Con questa dicitura non posso prendere numeri da 10 in su.

Negazione:

se all'interno delle parentesi quadre scriviamo

- `[^abc]` : **matchiamo qualsiasi carattere eccetto a,b o c** (il simbolo `^` dentro le parentesi quadre significa "not").

Si può combinare le classi:

Se si scrive

`[a-zA-Z0-9_]` : significa che in quella particolare posizione matcha qualsiasi lettere minuscola o maiuscola, cifra o underscore.

Shortcut delle RegEx

Siccome definire queste classi è lungo e complesso e può portare spesso a degli errori di sintassi ci sono degli shortcut:

- `\d` :
matcha un qualsiasi singola cifra (equivalente a `[0-9]`).
Anche se è non sono del tutto equivalenti in tutti i contesti; il `\d` matcha tutte le cifre Unicode e quindi è più lento nella verifica, tuttavia in Python, di default è come `[0-9]` .
- `\D` : matcha qualsiasi carattere che non sia un numero (equivalente a `[0-9]`).
- `\w` : prende qualsiasi lettere cifra, lettere o underscore (equivale a `[a-zA-Z0-9_]`)

- `\W` : nega tutti i caratteri che non sono caratteri
- `\s` : matcha qualsiasi spazio, tab, newline, etc.
- `\S` : matcha qualsiasi carattere che non è uno spazio, tab, newline, etc.

Boundary

- `\b`– Word boundary:

serve per creare limiti di parola, matcha la posizione tra un carattere di parola e un non carattere di parola.

Cioè Matcha la **posizione** tra un carattere di parola (`\w`) e un non-carattere (spazi, punteggiatura, ecc.).

Utile per cercare parole intere.

Supponiamo di matchare la parola cat, solo ed esclusivamente quella parola, per fare ciò scriviamo `\bcatt\` prende esattamente la parola "cat" della frase "the cat sat" ma non matcha "cat" della frase "caterpillar".

Quindi non si può usare per trovare le sotto-stringhe

≡ Title

supponiamo di avere un codice di quattro cifre consecutive



Python

```
1 "12345650459405"
2 #se scrivo:
3 [0-9]{4}
4 #io becco tutta la sotto stringa dei primi 4 caratteri, se
  invece metto un boundary gli dico che voglio una parola di
  solo 4 caratteri
5 \b[0-9]{4}\b
6 [0-9]{4}\b
7 \b[0-9]{4}
```

Un esempio di caso di applicazione può essere per trovare le prime cifre di un codice fiscale o di un numero di matricola

- `\B` : **Non-word boundary**
 - **matcha la posizione che non è un limite di parola, quindi trova anche le sotto-stringhe.**

Difatti è utile per trovare le sottostringhe nelle parole

Esempio:

- `\Bcat\B` matcha le parole "educate" o "catfish" ma non "cat", questo perché "cat" è una parola a se mentre "educate" o "catfish" contengono la sottostringa "cat" .

Usare le RegEx in Python

Per usare le RegEx in [Python](#), bisogna importare il modulo `re` : questo modulo ci permette di utilizzare tutti i metodi delle RegEx.

Esempio:



Python

```
1 import re
2 text: str = "My email is marco@gmail.com"
3 result: list[str] = re.findall(r'\S+@\S+', text)
4 print(result) # Output ['marco@gmail.com']
```

`findall()` :

Cerca tutte le occorrenze non sovrapposte (non-overlapping) del pattern nella string e le restituisce come lista di stringhe.

La regex(`\S+@\S+`):

1. `\S+`
 - `\S` = **Qualsiasi carattere che NON sia uno spazio bianco** (no spazi, tab, newline).
 - `+` = **1 o più occorrenze** dell'elemento precedente (quindi 1+ caratteri non-spazio).
 2. `@` :
 - Matcha il carattere `@` letterale (la "chiocciola").
 3. `\S+` :
 - Altri 1+ caratteri non-spazio dopo la `@` (es. `gmail.com`).
- **Trappole:**
 - Se l'email è seguita da un segno di punteggiatura (es. `marco@gmail.com,`), questo verrà incluso nel match (perché `,` non è uno spazio).
 - Non verifica la validità dell'email (es. `abc@xyz` sarebbe considerato valido).

Da notare una cosa: nel caso in cui si debba definire una regex formalmente, ovvero non si sa come deve venire usata dobbiamo scrivere esattamente come è scritta una mail, quindi la regex scritta sopra a livello formale è incorretta. Quindi bisogna suddividere il problema in 2 sotto problemi

1. Individuare quali sono i caratteri prime della @ :

Prima della chiocciola si possono trovare lettere (A-Za-z), numeri(0-9) e caratteri speciali come . , _ , % , + , - .

2. Individuare i caratteri dopo la @ :

Dopo la chiocciola in una email si ha un dominio valido(es: gmail.com) con:

- Lettere, numeri, punti . e trattini - .
- Un **TLD** (Top-Level Domain) di almeno 2 caratteri (es. .com , .io).

Quindi la regex può essere riscritta così:



Python

```
1 import re
2
3 regex = r'[\w.%+-]+@[ \w.-]+\.[a-zA-Z]{2,}'
4 text = "Email: marco.polo+test@gmail.com, a@b.co, invalid@xyz"
5 result = re.findall(regex, text)
6 print(result) # Output: ['marco.polo+test@gmail.com', 'a@b.co']
```

Spiegazione:

- `[\w.%+-]+` :
 - `\w` = lettere, numeri, underscore (_).
 - `. % + -` = caratteri speciali consentiti prima della @ .
- `@[\w.-]+` :
 - Matcha il dominio (es. gmail. o sub.domain.).
 - `\w` = lettere/numeri, . e - consentiti.
- `\.[a-zA-Z]{2,}` :
 - Verifica che il TLD sia almeno 2 lettere (es. .com , .io).

Esempio 1: Stringhe che iniziano con le lettere maiuscole

Supponiamo di voler sapere se una stringa inizi con una lettera maiuscola:



Python

```

1 import re
2 text:str = "Rome Paris"
3 result = re.match(r'[A-Z][a-z]+', text)
4 print(result.group()) # Output "Rome"

```

Come si può vedere, sia in questo caso che nel [caso precedente](#), per ogni situazione si utilizza un metodo specifico in base a ciò che si vuole ottenere.

Da notare che alla variabile `result` non è stata aggiunta la tipizzazione esplicita, poiché i metodi del modulo `re` restituiscono tipi diversi a seconda della funzione usata: alcuni restituiscono oggetti `re.Match` (come `match()` o `search()`), altri liste di stringhe (come `findall()`), o altri tipi ancora. La tipizzazione dipende quindi dal metodo specifico chiamato

Questa espressione regolare significa:

- `[A-Z]` : fa match con una singola lettera maiuscola
- `[a-z]+` : fa match con una o più lettere minuscole
- `result.group()` : in questo contesto restituisce l'intero match trovato, poiché **non ci sono gruppi catturanti espliciti** (cioè non viene usata la sintassi con le parentesi tonde `()`).

Esempio 3: Trovare i numeri

Supponiamo di voler **estrarre tutti i numeri presenti in una stringa**:



Python

```

1 import re
2 text = "I have 20 cats and 3 dogs"
3 numbers = re.findall(r'\d+', text)
4 print(numbers)

```

Spiegazione:

- `re.findall(r'\d+', text)` :
 - **Il metodo `findall()` cerca tutte le occorrenze che corrispondono al pattern fornito e le restituisce come una lista di stringhe.**
 - il pattern `\d+` significa:
 - `\d` : una cifra numerica (equivalente a `[0-9]`)
 - `+` : una o più occorrenze consecutive
- Quindi `\d+` trova sequenze di una o più cifre (numeri interi).

In questo caso, il testo contiene **due numeri**: 20 e 3.

Il metodo `findall()` li individua entrambi e li restituisce come lista di stringhe: `['20', '3']`.

Nota:

I numeri trovati sono stringhe e non interi, nel caso in cui servissero come interi li si può convertire usando il `map()`.



Python

```
1 numbers = list(map(int, numbers))
```

Il modulo `re` di Python: metodi comuni

1. `re.match(pattern, strings)` :

- **Descrizione:** Cerca una corrispondenza solo all'inizio della stringa
- **Restituisce:** Oggetto `Match` o `None`
- Esempio:



Python

```
1 re.match(r"\d+", "123abc") # Match: '123'
2 re.match(r"\d+", "abc123") #none (non inizia con un numero)
```

- ✓ Funziona perché la stringa inizia con un numero

2. `re.search(pattern, string)` :

- **Descrizione:**
Cerca la prima corrispondenza nella stringa, in qualsiasi punto.
- **Restituisce:** oggetto `Match` o `None`
- Esempio:



Python

```
1 re.search(r"\d+", "abc123") # Match: '123'
```

- ✓ Funziona anche se il numero è alla fine.

3. `re.fullmatch(pattern, string)`

- **Descrizione:** Verifica se tutta la stringa corrisponde esattamente al pattern.
In altre parole controlla se l'intera stringa matcha una specifica **RegEx**, quindi se si inserisce una stringa con solo cifre se la trova la ritorna, o per meglio dire:
Restituisce `Match` solo se l'intera stringa soddisfa il pattern, altrimenti `None` ****
- **Restituisce:** Oggetto `Match` o `None`
- Esempio:



Python

```
1 re.fullmatch(r"\d+", "123") # Match: '123'
```

- ✓ Funziona perché la stringa contiene solo numeri
- ✗ Fallisce con `"123abc"` perché la stringa contiene anche lettere.

4. `re.findall(pattern, string)`

- **Descrizione:** Ritorna una lista con tutte le corrispondenze non sovrapposte.
- **Restituisce:** Lista di stringhe
- Esempio:



Python

```
1 re.findall(r'\d\d', "12345")
2 #output ['12', '34']
```

- ✓ Trova tutti i match, ma ignora i casi sovrapposti
- ✓ Ogni numero viene trovato e aggiunto alla lista(nel caso di gruppi ()) restituisce una lista di tuple)

5. `re.finditer(pattern, string)`

- **Descrizione:**
Ritorna un iteratore di oggetti `Match` per tutte le corrispondenze.
Utile per usare le **RegEx** nei cicli `for`, accedendo anche a dettagli come posizione e contenuto.
- **Restituisce:** Iteratori di oggetti `match`.
- Esempio:



Python

```
1 for m in re.finditer(r"\d+", "a1b2"):
2     print(m.group()) # Output: 1, poi 2
```

✓ **Utile se si vuole accedere a posizione, contenuto, etc .**

Non è utile solo per i cicli, lo si può anche convertire in lista:



Python

```
1 list(re.finditer(r"\d+", "a1b2")) # Lista di oggetti Match
```

6. `re.sub(pattern, replacement, string)`

- **Descrizione:**

Sostituisce le parti che fanno match con un'altra stringa. In altre parole permette di rimpiazzare i match con altre stringhe, qui è l'unico caso in cui si ha una variazione.

- **Restituisce:** Una nuova stringa con le sostituzioni.
- **Esempio:**



Python

```
1 re.sub(r"\d", "#", "a1b2") # Output: 'a#b#'
```

✓ Ogni cifra viene rimpiazzata da # .

7. `re.split(pattern, string)`

- **Descrizione:**

Divide la stringa dove trova un match e ritorna una lista di parti. In altre parole suddivide una stringa in base al match, quindi separa le occorrenze e le restituisce come se fossero elementi singoli di una lista.

- **Restituisce:** Lista di stringhe
- **Esempio:**



Python

```
1 re.split(r"\d+", "a1b2c3") # Output: ['a', 'b', 'c', '']
```

✓ La divisione avviene **nei punti dove c'erano i numeri.**

Infatti l'ultimo elemento vuoto (' ') corrisponde al numero finale 3

8. `re.compile(pattern)`

- **Descrizione:**

Compila il pattern in un oggetto `RegexObject` per riutilizzarlo **efficientemente**, soprattutto in loop o chiamate ripetute.

In altre parole è utile per migliorare le prestazioni o per chiarezza nel codice, evitando di riscrivere la RegEx.

- **Restituisce:** Oggetto `Regex` compilato.
- Esempio:



Python

```
1 pattern = re.compile(r"\d+")
2 pattern.findall("a1b2") # Output: ['1', '2']
```

Esempi pratici: Matching con il Capture Groups

Ritornando al discorso dei gruppi e del non capturing a livello pratico:

In questo esempio possiamo vedere il grouping con capture con il `'re.search()'`.



Python

```
1 import re
2 text = "Il codice è:123-ABC"
3 match = re.search(r"(\d+)-([A-Z]+)", text)
4
5 if match:
6     print("Intera corrispondenza:", match.group(0)) # Output:
7     '123-ABC'
8     print("Gruppo 1 (numeri):", match.group(1)) #
9     Output: '123'
10    print("Gruppo 2 (lettere):", match.group(2)) #
11    Output: 'ABC'
```

Spiegazione:

- `()` : definisce **gruppi di cattura(capturing groups)**.
- `group(0)` : restituisce **l'intera corrispondenza** trovata dal pattern.
- `group(1)` e `group(2)` : restituiscono **ciò che è stato catturato nei singoli gruppi**, nell'ordine in cui compaiono nel pattern.
- Il `+` dopo `\d` e `[A-Z]` indica **"una o più occorrenze"**.

Esempio pratico: sostituzione con il Capture Groups

Il capture non è utile solo per lavorare sui singoli gruppi ma anche per mettere i gruppi nelle regex stesse:



Python

```
1 import re
2 text = "123-ABC"
3 new_text = re.sub(r"(\d+)-([A-Z]+)", r"\2-\1", text)
4 print(new_text) # Output: "ABC-123"
```

Spiegazione:

- In `re.sub` si può riutilizzare i gruppi nel replacement usando `\1`, `\2`, etc.
- Qui `\2` (le lettere) viene messo prima di `\1` (i numeri), **invertendo i due gruppi**.

Esempio pratico: il Non-Capturing



Python

```
1 import re
2 text = "abcabcabc"
3
4 print("Cattura:", re.findall(r"(abc)+", text)) # Output:
# ['abc'] (solo l'ultimo match del gruppo)
5 print("Non cattura:", re.findall(r"(?:abc)+", text)) # Output:
# ['abcabcabc'] (intera corrispondenza)
```

Spiegazione:

- `(abc)+` è un gruppo di cattura, quindi `findall()` restituisce solo l'ultimo match del gruppo catturato, non tutta la sequenza.
- `(?:abc)+` è un gruppo non-capturing (usato solo per raggruppare ma senza catturare): `findall()` allora restituisce l'intera corrispondenza del pattern, non solo del gruppo.

⚠ **Questo comportamento è specifico di `findall()`: restituisce solo i gruppi catturati se presenti; altrimenti restituisce l'intero match.**

🔗 Differenza chiave tra capturing groups e il non-capturing groups

🔗 Il capturing groups(`()`) viene usato solo quando si necessita di estrarre le parti del match

🔗 Il non-capturing groups(`?:...`) si usa solo quando si necessita di raggruppare per ripetizioni o logica; **non per ritornare i dati**

☰ In sintesi

Capture groups: Usa `()` se ti serve **estrarre**

Non-Capture groups: usa `?:...` se ti serve **solo raggruppare** per logica o ripetizione.

[regex101](#)